

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación

Proyecto 3

Simulador de Sistema de Archivos Virtual

Documentación Técnica

Estudiante: **Jordano Escalante**

Profesor: **Cristian Campos**

Curso: **IC-6600 Principios de Sistemas Operativos**

Semestre: **I Semestre 2026**

Contenido

1. Introduccion.....	3
2. Arquitectura del Sistema.....	4
3. Arranque del Programa.....	6
4. Descripcion de Clases.....	8
4.1 Paquete filesystem.....	9
4.2 Paquete storage.....	11
4.3 Paquete security.....	12
4.4 Paquete users.....	13
4.5 Paquete terminal.....	14
4.6 Paquete commands.....	15
5. Flujos de Ejecucion.....	16
6. Manual de Usuario.....	20
6.1 Arranque y entorno de ejecucion.....	20
6.2 Gestion de terminales multiples.....	20
6.3 Referencia de comandos.....	20
7. Bibliografia.....	30

1. Introduccion

1.1 Descripcion General

El VFS Simulator es un simulador de sistema de archivos virtual implementado en Java que emula las estructuras y operaciones fundamentales de un sistema de archivos real operando sobre consola interactiva. El sistema gestiona un disco virtual persistido en el archivo `miDiscoDuro.fs` mediante serializacion Java, soporta multiples sesiones de terminal de forma simultanea y aplica un modelo de control de acceso basado en propietario y grupo, analogo al modelo UNIX.

El proyecto fue desarrollado en el contexto del curso IC-6600 Principios de Sistemas Operativos del Instituto Tecnologico de Costa Rica, con referencia a los conceptos teoricos expuestos en Silberschatz et al. (Operating System Concepts) y Stallings (Operating Systems: Internals and Design Principles).

1.2 Objetivos

- Implementar las estructuras de datos centrales de un sistema de archivos: MBR, SuperBloque, FCB (File Control Block), mapa de bits de espacio libre y tabla de archivos abiertos (OFT).
- Simular el ciclo completo de vida de un archivo: creacion, apertura, lectura, escritura, modificacion de permisos y eliminacion con liberacion de bloques.
- Aplicar control de acceso estilo UNIX con clases propietario y grupo, con soporte de permisos `r/w/x` por clase.
- Soportar multiples terminales concurrentes sobre un mismo disco virtual compartido.
- Persistir el estado del sistema de archivos entre sesiones mediante serializacion Java.

1.3 Alcance

El simulador implementa un subconjunto funcional de las operaciones de un sistema de archivos UNIX. No implementa manejo real de hardware, interrupciones ni planificacion de procesos. La concurrencia entre terminales es simulada mediante un unico hilo con alternancia de sesiones, por lo que la tabla de archivos abiertos no refleja apertura simultanea real. Los limites de almacenamiento son determinados por el tamano de disco definido al ejecutar el comando `format`.

1.4 Entorno de Ejecucion

Desarrollado en Java 17+. No requiere dependencias externas. El archivo compilado se ejecuta desde la linea de comandos de cualquier sistema operativo con JRE disponible.

EJECUCION

```
java -jar vfs.jar          # primer arranque (sin disco previo)
java -jar vfs.jar          # arranques subsiguientes (carga miDiscoDuro.fs)
```

2. Arquitectura del Sistema

2.1 Estructura de Paquetes

El proyecto se organiza en seis paquetes funcionales mas la clase de entrada principal:

myFileSystem.java	(entrada principal, paquete por defecto)
filesystem/	(estructuras del sistema de archivos)
VirtualDisk.java	disco virtual, punto central de coordinacion
FCB.java	File Control Block / inodo
SuperBlock.java	metadatos de volumen (magic, nombre, bloques)
MBR.java	Master Boot Record
DirectoryEntry.java	entrada nombre<->FCB dentro de un directorio
OpenFileTable.java	tabla global de archivos abiertos
OpenFileEntry.java	una entrada en la OFT
storage/	(gestion de bloques y persistencia)
FreeSpaceManager.java	mapa de bits de bloques libres
BlockManager.java	leer/escribir contenido en bloques fisicos
DiskPersistence.java	serializacion/deserializacion del disco
security/	(control de acceso)
AccessControl.java	fachada publica de verificacion de permisos
PermissionManager.java	logica de bits r/w/x por clase
PasswordHasher.java	hash SHA-256 para contraseñas
users/	(gestion de identidades)
User.java / Group.java / UserManager.java	
terminal/	(interfaz de usuario)
Terminal.java	sesion individual de shell
TerminalManager.java	gestor de multiples sesiones
commands/	(27 clases, una por comando)
Command.java	interfaz comun
FormatCommand.java ...	implementaciones

2.2 Diagrama de Dependencias

Las clases de mayor centralidad son VirtualDisk (referenciada por casi todos los componentes), FCB (nucleo de metadatos) y Terminal (coordinacion entre entrada del usuario y ejecucion de comandos):

```

myFileSystem
|-- DiskPersistence.load() --> VirtualDisk (objeto serializado)
|-- new TerminalManager(disk, diskFile)
    |-- new Terminal (1..N)
        |-- registerCommands() (27 objetos Command)
        |-- executeCommandLine(line)
            |-- cmd.execute(args, terminal)
                |-- VirtualDisk
                    |-- FCB (fcbTable)
                    |-- FreeSpaceManager
                    |-- OpenFileTable
                    |-- UserManager
                    |-- SuperBlock / MBR / blocks[]
                |-- BlockManager --> VirtualDisk
                |-- AccessControl --> PermissionManager
                |-- DiskPersistence (via saveDisk)

```

2.3 Modelo de Datos en Memoria

Al estar cargado, un VirtualDisk mantiene en heap Java las siguientes estructuras:

Estructura	Tipo Java	Descripcion
SuperBlock / MBR	Objetos Serializable	Metadatos del volumen: nombre, tamaño, magic, conteo de bloques
blocks	byte[][]	blocks[i] = bloque i de 512 bytes. Datos brutos del "disco"
fcbTable	Map<String, FCB>	UUID -> FCB. Contiene TODOS los archivos, dirs y enlaces
freeSpaceManager	FreeSpaceManager	boolean[] bitmap. bitmap[i]=true indica bloque ocupado
userManager	UserManager	Map<String,User> y Map<String,Group> en memoria
openFileTable	OpenFileTable	List<OpenFileEntry> con archivos actualmente abiertos

Todas implementan Serializable, permitiendo que DiskPersistence serialice el grafo completo a un unico archivo .fs mediante ObjectOutputStream en un solo paso.

3. Arranque del Programa

El ciclo de arranque comienza en `myFileSystem.main()` y sigue la siguiente secuencia de interacciones entre clases:

Paso 1 — Entrada en `main()`

`myFileSystem.main(String[] args)` determina el nombre del archivo de disco: si `args[0]` existe usa ese valor; en caso contrario usa el literal `"miDiscoDuro.fs"`.

Paso 2 — Deteccion del disco existente

Se invoca `DiskPersistence.diskExists(diskFile)`, que comprueba si el archivo existe en el directorio de trabajo. Si existe, se llama `DiskPersistence.load(diskFile)`:

- `load()` abre el archivo mediante `FileInputStream` envuelto en `ObjectInputStream`.
- Deserializa el objeto `VirtualDisk` completo con todo su grafo de objetos (FCBs, bloques, usuarios, OFT).
- Retorna el `VirtualDisk` restaurado o null en caso de error (archivo corrupto, version incompatible de la JVM).
- Si `load()` retorna un `VirtualDisk` valido, se verifica `disk.getSuperBlock().isValid()`, que comprueba `magic == 0xCAFEBAE`. Si falla o no existe disco, `disk` queda en null.

Paso 3 — Creacion del `TerminalManager`

Se construye `new TerminalManager(disk, diskFile)`. El constructor:

- Almacena la referencia al disco (puede ser null si no habia disco previo).
- Crea el primer `Terminal`: `new Terminal(1, disk, diskFile)`.
- El constructor de `Terminal` llama `registerCommands()`, que instancia los 27 objetos `Command` y los registra en un `LinkedHashMap<String, Command>` indexado por nombre del comando.

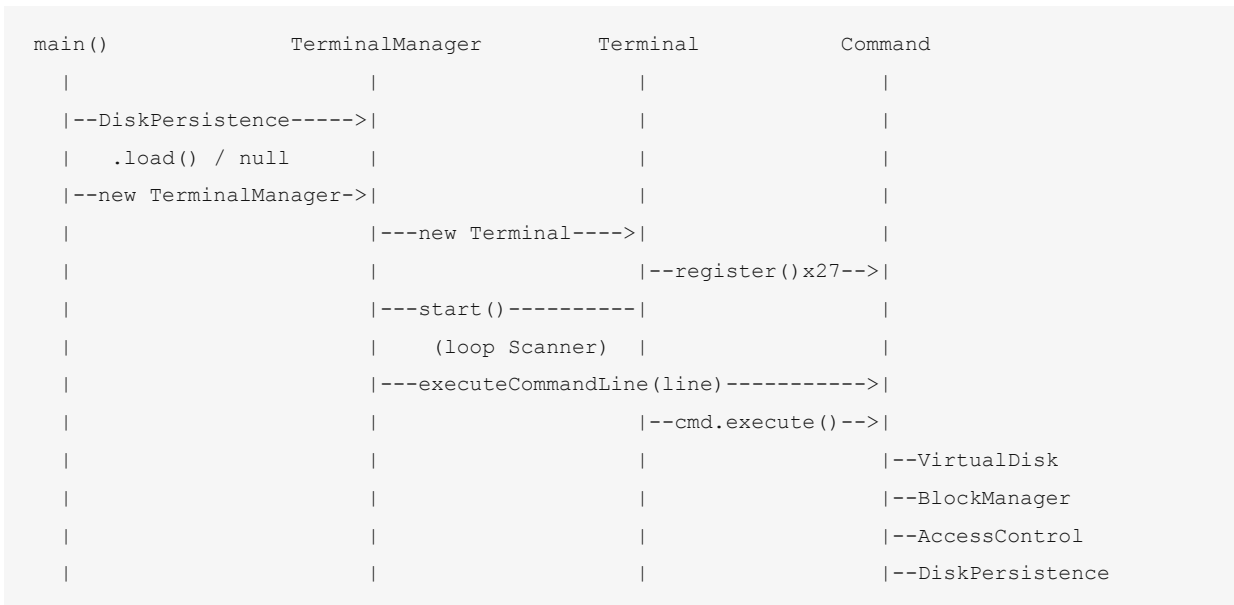
Paso 4 — Loop principal (`TerminalManager.start()`)

- Imprime el banner con `printBanner()`.
- Crea un unico `Scanner` sobre `System.in` y lo inyecta en todos los terminales via `terminal.setScanner(sc)`. Un solo `Scanner` evita conflictos de buffering al leer lineas desde multiples lectores sobre el mismo `InputStream`.
- Entra en el bucle `while(true)`: lee una linea, determina si es meta-comando (`newterminal`, `switcht N`, `terminals`, `closeterminal N`) o lo delega al terminal activo via `active.executeCommandLine(line)`.
- Despues de cada comando, `syncDisk(active)` propaga un nuevo `VirtualDisk` si el terminal ejecuto `format`.

Paso 5 — Despacho en `Terminal.executeCommandLine()`

- `tokenize(line)` divide la linea por espacios, respetando cadenas entre comillas dobles.
- Verifica si el comando requiere login: `format`, `exit`, `su`, `help`, `clear`, `infofs` no lo requieren.
- Verifica si el comando requiere disco activo: `format`, `exit`, `help`, `clear` no lo requieren.
- Busca el Command en el mapa: `commands.get(name.toLowerCase())`.
- Llama `cmd.execute(tokens, this)` pasando el arreglo de tokens y la referencia al Terminal como contexto.

DIAGRAMA DE SECUENCIA DE ARRANQUE



4. Descripcion de Clases

Esta seccion documenta el proposito, atributos y metodos de cada clase del sistema, organizadas por paquete.

4.1 Paquete filesystem

VirtualDisk (implements Serializable)

Clase central del sistema. Encapsula la totalidad del estado del disco virtual: bloques de datos, metadatos de volumen, tabla de FCBs, espacio libre, tabla de archivos abiertos y base de datos de usuarios. Es el unico objeto que se serializa al archivo .fs; todos los demas son alcanzables desde el.

Campo	Tipo	Descripcion
BLOCK_SIZE	static final int	Tamano de bloque = 512 bytes
mbr	MBR	Master Boot Record del volumen
superBlock	SuperBlock	Metadatos del volumen (magic, nombre, bloques libres)
freeSpaceManager	FreeSpaceManager	Mapa de bits de bloques libres
fcbTable	Map<String,FCB>	UUID -> FCB. Todos los archivos, directorios y enlaces
blocks	byte[][]	blocks[i] = datos crudos del bloque i (512 bytes)
userManager	UserManager	Base de datos de usuarios y grupos
openFileTable	OpenFileTable	Tabla de archivos actualmente abiertos
rootFcbId	String	UUID del FCB del directorio raiz /

Metodo	Descripcion
readBlock(int i)	Retorna copia del bloque i (byte[512])
writeBlock(int i, byte[])	Escribe datos en el bloque i
putFCB(FCB)	Agrega o reemplaza FCB en fcbTable por su UUID
getFCBByPath(String)	Busca FCB cuyo campo path coincida exactamente
getParentFCB(String)	Extrae directorio padre del path y llama getFCBByPath
removeFCB(String uuid)	Elimina el FCB de fcbTable
normalizePath(String)	Resuelve ".." y "." para obtener ruta absoluta canonica
syncSuperBlock()	Actualiza SuperBlock.freeBlocks desde FreeSpaceManager.getFreeCount()

FCB — File Control Block (implements Serializable)

Equivalente al inodo en sistemas UNIX. Cada archivo, directorio o enlace simbólico tiene exactamente un FCB. El ID es un UUID generado al construirse y no cambia. El sistema de permisos usa un entero de dos digitos: decenas = bits del propietario, unidades = bits del grupo. Cada digito suma los bits activos (r=4, w=2, x=1); por ejemplo 74 = propietario rwx (7), grupo r-- (4).

Campo	Tipo	Descripcion
id	String (UUID)	Identificador unico generado con UUID.randomUUID()
name	String	Nombre del archivo sin ruta
path	String	Ruta absoluta completa
owner / group	String	Propietario y grupo del archivo
permissions	int	Permisos en formato 2 digitos (ej: 77, 74, 64)
size	long	Tamano en bytes del contenido
creationDate / modificationDate	Date	Fechas de creacion y ultima modificacion
isDirectory	boolean	True si el FCB representa un directorio
isSymlink / linkTarget	boolean / String	True y ruta destino si es enlace simbolico
blocks	List<Integer>	Indices de bloques asignados (asignacion indexada)
entries	List<DirectoryEntry>	Entradas del directorio (null si no es directorio)

Metodo	Descripcion
<code>getPermissionsString()</code>	Convierte el entero de permisos a cadena rwx, ej: 74 -> "rwxr--"
<code>typeTag()</code>	Retorna [F], [D] o [L] segun el tipo del FCB
<code>addEntry(DirectoryEntry)</code>	Agrega una entrada al directorio (requiere <code>isDirectory=true</code>)
<code>removeEntry(String name)</code>	Elimina la entrada con ese nombre del directorio
<code>findEntry(String name)</code>	Busca y retorna la entrada por nombre, o null

SuperBlock (implements Serializable)

Almacena los metadatos de volumen. Se valida al cargar el disco comparando el campo `magic` con `FS_MAGIC = 0xCAFEBAE`. `syncSuperBlock()` debe llamarse antes de serializar para que `freeBlocks` sea preciso.

Campo	Tipo	Descripcion
<code>FS_MAGIC</code>	static final int	0xCAFEBAE — magic number de validacion del FS
<code>magic</code>	int	Valor almacenado; debe igualar <code>FS_MAGIC</code> para considerarse valido
<code>fsName</code>	String	Nombre del volumen
<code>totalSize</code>	long	Tamano total del disco en bytes
<code>blockSize</code>	int	Tamano de bloque = 512 bytes
<code>totalBlocks / freeBlocks</code>	int	Total de bloques y bloques actualmente libres
<code>creationDate</code>	Date	Fecha de creacion del volumen

MBR — Master Boot Record (implements Serializable)

Emula el registro de arranque maestro. Contiene la firma "MBR-VFS-1.0", nombre del volumen, tamano total y parametros de la unica particion. `isValid()` compara el campo `signature` con la constante `SIGNATURE`.

DirectoryEntry (implements Serializable)

Asociacion nombre <-> FCB dentro de un directorio. El campo `name` es el nombre visible para el usuario; `fcblid` es el UUID del FCB al que apunta. Un directorio es un FCB con `isDirectory=true` cuya lista `entries` contiene objetos `DirectoryEntry`.

OpenFileTable y OpenFileEntry (implements Serializable)

`OpenFileTable` es la tabla global de archivos abiertos del sistema (OFT). Cuando un comando abre un archivo llama `open(fcb, username)`, que crea una `OpenFileEntry` o incrementa el `referenceCount` si ya existia. Al cerrar con `close()` se decrementa el contador; cuando llega a cero la entrada se elimina y `fcb.setOpen(false)`.

Campo de OpenFileEntry	Tipo	Descripcion
<code>fcblid</code>	String	UUID del FCB abierto
<code>filename / filepath</code>	String	Nombre y ruta absoluta del archivo
<code>openedByUser</code>	String	Usuario que ejecuto la apertura
<code>openDate</code>	Date	Momento en que se abrio el archivo
<code>referenceCount</code>	int	Numero de referencias activas sobre este FCB

4.2 Paquete storage

FreeSpaceManager (implements Serializable)

Gestiona el espacio libre del disco mediante un mapa de bits. Cada posicion *i* del arreglo `boolean[] bitmap` corresponde al bloque *i*: `false` = libre, `true` = ocupado.

Campo	Tipo	Descripcion
<code>bitmap</code>	<code>boolean[]</code>	<code>bitmap[i]=true</code> indica bloque <i>i</i> ocupado
<code>totalBlocks</code>	<code>int</code>	Total de bloques del disco
<code>freeCount</code>	<code>int</code>	Contador de bloques actualmente libres

Metodo	Descripcion
<code>allocate(int count)</code>	Encuentra <code>count</code> bloques libres; retorna <code>List<Integer></code> con indices o <code>null</code> si no hay espacio
<code>allocateOne()</code>	Reserva un unico bloque; retorna su indice o <code>-1</code> si disco lleno
<code>free(List<Integer>)</code>	Libera los bloques indicados poniendo <code>bitmap[i]=false</code> , incrementa <code>freeCount</code>
<code>freeOne(int idx)</code>	Libera un unico bloque
<code>getFreeCount()</code>	Cantidad de bloques actualmente libres
<code>getBitmapString()</code>	Cadena visual de 0/1 (max 80 chars) para diagnostico con <code>infos</code>

BlockManager (clase utilitaria estatica)

Abstrae la lectura y escritura de contenido de texto sobre el array de bloques del `VirtualDisk`. No mantiene estado propio (constructor privado).

Metodo	Descripcion
<code>readContent(disk, fcb)</code>	Itera <code>fcb.getBlocks()</code> , llama <code>disk.readBlock(i)</code> por cada uno, concatena y retorna <code>String</code>
<code>writeContent(disk, fcb, cont)</code>	(1) Libera bloques actuales con <code>FreeSpaceManager.free()</code> . (2) Convierte <code>content</code> a bytes. (3) Calcula bloques
<code>releaseBlocks(disk, fcb)</code>	Libera todos los bloques del FCB sin escribir contenido (usado por <code>rm</code>)
<code>initEmpty(fcb)</code>	Establece lista de bloques vacia en el FCB para archivos nuevos con <code>touch</code>

DiskPersistence (clase utilitaria estatica)

Maneja la persistencia del `VirtualDisk`. Antes de serializar, `save()` llama `disk.syncSuperBlock()` para que el conteo de bloques libres este actualizado. Usa `ObjectOutputStream/ObjectInputStream` de `java.io`.

Metodo	Descripcion
<code>save(disk, filePath)</code>	Sincroniza <code>SuperBlock</code> , serializa el disco completo con <code>ObjectOutputStream</code>
<code>load(filePath)</code>	Deserializa y retorna el <code>VirtualDisk</code> , o <code>null</code> si el archivo no existe o esta corrupto
<code>diskExists(filePath)</code>	Retorna <code>true</code> si el archivo existe en el sistema de archivos real
<code>fileSize(filePath)</code>	Retorna el tamano del archivo serializado en bytes

4.3 Paquete security

PermissionManager (clase utilitaria estatica)

Implementa la logica de evaluacion de bits de permiso. Las constantes READ=4, WRITE=2, EXECUTE=1 se corresponden con los bits del digito de permiso. El metodo principal hasAccess() sigue este algoritmo:

- 1. Si username == "root": retorna true siempre. Root no esta sujeto a restricciones de permisos.
- 2. Si username == fcb.getOwner(): evalua el digito de las decenas de fcb.getPermissions() con (digit & requiredBit) != 0.
- 3. Si UserManager.isMemberOf(username, fcb.getGroup()): evalua el digito de las unidades de igual forma.
- 4. En cualquier otro caso: retorna false. No existe clase "others" en este sistema.

isValidPermission(int perms) verifica que perms sea de dos digitos y que cada digito este en el rango 0..7.

AccessControl (clase utilitaria estatica)

Fachada publica sobre PermissionManager. Expone checkRead(), checkWrite() y checkExecute(). En caso de denegacion imprime un mensaje de error descriptivo. Todos los comandos usan AccessControl, nunca PermissionManager directamente.

PasswordHasher (clase utilitaria estatica)

Genera y verifica hashes SHA-256 en hexadecimal mediante MessageDigest("SHA-256"). Las contraseñas NUNCA se almacenan en texto plano. El campo passwordHash del objeto User siempre contiene el hash. verify(plaintext, storedHash) calcula el hash del texto plano y lo compara.

4.4 Paquete users

User (implements Serializable)

Representa una cuenta de usuario. `isRoot` se establece en `true` si y solo si `username == "root"`. La contraseña se almacena como hash SHA-256; nunca en texto plano.

Campo	Tipo	Descripcion
<code>username</code>	<code>String</code>	Nombre de inicio de sesion
<code>fullName</code>	<code>String</code>	Nombre completo (informativo)
<code>passwordHash</code>	<code>String</code>	Hash SHA-256 de la contraseña
<code>primaryGroup</code>	<code>String</code>	Nombre del grupo principal del usuario
<code>homeDirectory</code>	<code>String</code>	Ruta home (<code>/root</code> o <code>/home/users/<usuario></code>)
<code>isRoot</code>	<code>boolean</code>	True si <code>username == "root"</code>

Group (implements Serializable)

Agrupar usuarios con derechos de acceso comunes. Contiene nombre y `List<String> members`. `addMember()` es idempotente. El FCB de cada archivo almacena el nombre del grupo propietario; `PermissionManager` consulta `UserManager.isMemberOf()` para verificar pertenencia al grupo.

UserManager (implements Serializable)

Gestiona la base de datos de usuarios y grupos. Mantiene `Map<String, User>` y `Map<String, Group>` en memoria. Es `Serializable` y se almacena dentro del `VirtualDisk`.

Metodo	Descripcion
<code>addUser(username, fullName, plainPwd, groupName)</code>	Hashea la contraseña, crea User, lo agrega al mapa y al grupo principal
<code>authenticate(username, plainPwd)</code>	Calcula hash del texto plano y compara con el hash almacenado
<code>changePassword(username, newPwd)</code>	Hashea la nueva contraseña y actualiza el campo <code>passwordHash</code>
<code>isMemberOf(username, groupName)</code>	True si <code>groupName</code> es el grupo principal o si <code>username</code> esta en <code>members</code> del grupo
<code>addGroup(groupName) / removeGroup()</code>	Crea o elimina grupo del mapa de grupos
<code>userCount() / groupCount()</code>	Retorna el numero de usuarios o grupos registrados

4.5 Paquete terminal

Terminal

Representa una sesion de shell independiente. Cada Terminal tiene su propio usuario activo (`currentUser`), directorio de trabajo (`currentDirectory`) y mapa de comandos. Todos los terminales comparten el mismo `VirtualDisk` y el mismo `Scanner` sobre `System.in`.

Campo	Tipo	Descripcion
<code>id</code>	<code>int</code>	Numero de terminal (T1, T2, ...)
<code>disk</code>	<code>VirtualDisk</code>	Referencia al disco virtual compartido
<code>currentUser</code>	<code>String</code>	Usuario activo; null indica que no hay sesion iniciada
<code>currentDirectory</code>	<code>String</code>	Ruta absoluta del directorio de trabajo actual
<code>running</code>	<code>boolean</code>	False cuando <code>exit</code> es ejecutado en esta terminal
<code>sharedScanner</code>	<code>Scanner</code>	Instancia unica de <code>System.in</code> inyectada por <code>TerminalManager</code>
<code>commands</code>	<code>Map<String,Command></code>	Mapa nombre->Command con los 27 comandos registrados

Metodo	Descripcion
<code>registerCommands()</code>	Instancia y registra los 27 objetos <code>Command</code> en el mapa <code>commands</code>
<code>executeCommandLine(String)</code>	Tokeniza la linea, aplica guards de login/disco, despacha al <code>Command</code>
<code>tokenize(String)</code>	Divide por espacios respetando cadenas entre comillas dobles
<code>resolvePath(String)</code>	Convierte rutas relativas, "~" y "." a ruta absoluta via <code>normalizePath()</code>
<code>getPrompt()</code>	Genera "[T<id>] <user>@vfs:<cwd>[#/\$] "
<code>saveDisk()</code>	Delega en <code>DiskPersistence.save(disk, diskFile)</code>

TerminalManager

Coordina multiples sesiones Terminal sobre un unico `Scanner`. Los meta-comandos son interceptados antes de llegar a cualquier Terminal.

Meta-comando	Comportamiento
<code>newterminal</code>	Crea Terminal con <code>id=max(ids)+1</code> , comparte el <code>Scanner</code> , lo activa
<code>switcht N</code>	Busca el terminal con <code>id==N</code> y actualiza <code>activeldx</code>
<code>terminals</code>	Lista terminales activas con usuario y directorio de cada una
<code>closeterminal N</code>	Elimina el terminal N. No funciona si es el unico terminal activo

Metodo	Descripcion
<code>start()</code>	Loop principal: lee <code>Scanner</code> , detecta meta-comandos, delega al terminal activo
<code>syncDisk(Terminal)</code>	Si el terminal formateo un nuevo disco, propaga la referencia a todos
<code>isMetaCommand(String)</code>	True si la linea es <code>newterminal</code> , <code>switcht</code> , <code>terminals</code> o <code>closeterminal</code>

4.6 Paquete commands

Interfaz Command

Define el contrato de todos los comandos. Tres metodos: `getName()` retorna el nombre clave para el despacho, `getDescription()` retorna descripcion textual, y `execute(String[] args, Terminal terminal)` contiene la logica. El primer elemento de `args` es siempre el nombre del comando.

Implementaciones

Clase	Nombre	Clases involucradas en execute()
FormatCommand	format	VirtualDisk, FreeSpaceManager, UserManager, DiskPersistence
ExitCommand	exit	Terminal.stop(), DiskPersistence.save()
SuCommand	su	UserManager.authenticate(), PasswordHasher
UseraddCommand	useradd	UserManager.addUser(), VirtualDisk (crear dir home)
GroupaddCommand	groupadd	UserManager.addGroup()
PasswdCommand	passwd	UserManager.authenticate(), changePassword()
WhoamiCommand	whoami	UserManager.getUser()
CdCommand	cd	VirtualDisk.getFCBByPath(), AccessControl.checkExecute()
PwdCommand	pwd	Terminal.getCurrentDirectory()
LsCommand	ls	FCB.getEntries(), AccessControl.checkRead()
TouchCommand	touch	VirtualDisk.putFCB(), FCB, BlockManager.initEmpty()
MkdirCommand	mkdir	VirtualDisk.putFCB(), FCB(isDirectory=true)
RmCommand	rm	BlockManager.releaseBlocks(), OpenFileTable, VirtualDisk
MvCommand	mv	FCB.setName()/setPath(), updateChildPaths(), VirtualDisk
LnCommand	ln	FCB(isSymlink=true, linkTarget), VirtualDisk.putFCB()
CatCommand	cat	OpenFileTable.open/close(), BlockManager.readContent()
LessCommand	less	BlockManager.readContent(), Scanner (paginacion 20 lineas)
NoteCommand	note	OpenFileTable, BlockManager.writeContent(), Scanner
ChmodCommand	chmod	FCB.setPermissions(), PermissionManager.isValidPermission()
ChownCommand	chown	FCB.setOwner(), UserManager.userExists()
ChgrpCommand	chgrp	FCB.setGroup(), UserManager.groupExists()
WhereisCommand	whereis	VirtualDisk.getFcbTable() (busqueda en tabla completa)
ViewFCBCommand	viewfcb	FCB (todos los atributos)
ViewFilesOpenCommand	viewfilesopen	OpenFileTable.getEntries()
InfoFSCommand	infofs	SuperBlock, MBR, FreeSpaceManager, UserManager, OpenFileTable
HelpCommand	help	CATALOG estatico con 26 entradas de ayuda
ClearCommand	clear	Secuencia ANSI \033[H\033[2J

5. Flujos de Ejecucion

Esta seccion describe paso a paso la interaccion entre clases durante la ejecucion de los comandos mas representativos. Cada flujo parte desde la recepcion de la linea de entrada en `TerminalManager.start()` y termina cuando el control regresa al prompt.

5.1 format — Creacion del disco virtual

Es el comando mas complejo: crea desde cero todas las estructuras del VFS.

- `FormatCommand.execute(["format"], terminal)`
- `parseSize(input)` convierte "10MB" -> $10 \times 1024 \times 1024 = 10,485,760$ bytes. Calcula `totalBlocks = floor(totalSize/512) = 20,480`.
- `new VirtualDisk(volumeName, totalSize):`
 - `new MBR(name, size, blocks) | new SuperBlock(name, size, 512)`
 - `new FreeSpaceManager(blocks) -> boolean[20480]` todos false
 - `new byte[20480][512]` -> todos ceros
 - `new UserManager() | new OpenFileTable()`
- Arbol de directorios: para cada ruta en `["/", "/root", "/home", "/home/users"]`:
 - `new FCB(name, "root", "root", 77, true, path)`
 - `disk.putFCB(fcb) -> fcbTable.put(uuid, fcb)`
 - `parent.addEntry(new DirectoryEntry(name, fcb.getId()))`
- Creacion de root: `UserManager.addGroup("root")` | `UserManager.addUser("root", "root", "root", "root", "/root")`
 - `PasswordHasher.hash("root") -> SHA-256 hex`
 - `new User("root", ..., hash, "root", "/root"), User.isRoot=true`
 - `Group("root").addMember("root")`
- `terminal.setCurrentUser("root") | terminal.setCurrentDirectory("/root")`
- `DiskPersistence.save(disk, "miDiscoDuro.fs")`:
 - `disk.syncSuperBlock() -> SuperBlock.freeBlocks = FreeSpaceManager.getFreeCount()`
 - `ObjectOutputStream.writeObject(disk) -> serializa todo el grafo en un solo archivo`
- `TerminalManager.syncDisk()` propaga el nuevo `VirtualDisk` a todos los terminales activos

5.2 note — Creacion y escritura de archivo

Ilustra el ciclo completo de creacion de FCB, registro en OFT, asignacion de bloques y persistencia.

- `NoteCommand.execute(["note", "texto.txt"], terminal)`
- `path = terminal.resolvePath("texto.txt") -> "/root/texto.txt"`
- `fcb = disk.getFCBByPath("/root/texto.txt") -> null` (archivo nuevo)
- `parent = disk.getParentFCB("/root/texto.txt") -> FCB del directorio "/root"`
- `AccessControl.checkWrite(parent, "root", disk) -> PermissionManager.hasAccess() -> isRoot=true -> true`

- new FCB("texto.txt", "root", "root", 77, false, "/root/texto.txt")
- id = UUID.randomUUID() -> "a3f7c921-..." | blocks = new ArrayList<>() | isOpen=false
- disk.putFCB(fcb) -> fcbTable.put(id, fcb)
- parent.addEntry(new DirectoryEntry("texto.txt", id))
- disk.getOpenFileTable().open(fcb, "root"):
- new OpenFileEntry(id, "texto.txt", "/root/texto.txt", "root")
- entries.add(entry) | fcb.setOpen(true)
- Bucle de editor: reads via terminal.getScanner().nextLine() hasta ":x"
- Usuario confirma con "Y" | content = "hola mundo" (10 bytes)
- BlockManager.writeContent(disk, fcb, "hola mundo"):
- FreeSpaceManager.free([]) -> sin efecto (lista vacia)
- bytes = content.getBytes() -> 10 bytes
- needed = ceil(10/512) = 1 | FreeSpaceManager.allocate(1) -> [5] -> bitmap[5]=true, freeCount--
- data[0..9] = bytes; data[10..511] = 0x00 (padding)
- disk.writeBlock(5, data) -> blocks[5] = data
- fcb.setBlocks([5]) | fcb.setSize(10) | fcb.setModificationDate(new Date())
- disk.syncSuperBlock() -> SuperBlock.freeBlocks actualizado
- disk.getOpenFileTable().close(id, "root", fcb):
- entry.decrementRef() -> refCount=0 -> entries.remove(entry) | fcb.setOpen(false)
- terminal.saveDisk() -> DiskPersistence.save()

5.3 cat — Lectura de archivo

- CatCommand.execute(["cat", "notas.txt"], terminal)
- path = terminal.resolvePath("notas.txt") -> "/root/notas.txt"
- fcb = disk.getFCBByPath(path)
- Si fcb.isSymlink(): path = fcb.getLinkTarget(); fcb = disk.getFCBByPath(path) (sigue el enlace)
- Si fcb == null despues de seguir enlace: System.out.println("cat: enlace roto"); return
- AccessControl.checkRead(fcb, user, disk) -> PermissionManager evalua bit READ sobre propietario/grupo
- disk.getOpenFileTable().open(fcb, user)
- content = BlockManager.readContent(disk, fcb):
- Para cada idx en fcb.getBlocks(): data = disk.readBlock(idx)
- Concatena todos los byte[], convierte a String y retorna
- System.out.println(content)
- disk.getOpenFileTable().close(fcbId, user, fcb)

5.4 rm — Eliminacion y liberacion de bloques

- RmCommand.execute(["rm", "archivo.txt"], terminal)
- Detecta si hay flag -R
- path = terminal.resolvePath("archivo.txt")

- target = disk.getFCBByPath(path)
- parent = disk.getParentFCB(path)
- AccessControl.checkWrite(parent, user, disk) -> verifica permiso de escritura en directorio padre
- Si OpenFileTable.isOpen(target.getId()): openFileTable.forceClose(id, target) -> forzar cierre
- BlockManager.releaseBlocks(disk, target):
 - FreeSpaceManager.free(target.getBlocks()) -> bitmap[5]=false, freeCount++
 - target.setBlocks(new ArrayList<>())
- parent.removeEntry("archivo.txt") -> elimina DirectoryEntry de la lista del padre
- disk.removeFCB(target.getId()) -> fcbTable.remove(id)
- disk.syncSuperBlock() | terminal.saveDisk()
- Con -R sobre directorio: removeDirectoryRecursive(fcb, disk) aplica el proceso anterior recursivamente a todos los FCBs descendientes antes de eliminar el directorio padre

5.5 chmod — Modificacion de permisos

- ChmodCommand.execute(["chmod", "74", "notas.txt"], terminal)
- newPerms = Integer.parseInt("74")
- PermissionManager.isValidPermission(74): cada digito en 0..7 -> true
- fcb = disk.getFCBByPath(terminal.resolvePath("notas.txt"))
- Verifica que currentUser == fcb.getOwner() OR currentUser == "root"
- oldPerms = fcb.getPermissions() -> 77
- fcb.setPermissions(74) | fcb.setModificationDate(new Date())
- Imprime: "Permisos actualizados: 77 -> 74 (rwxr--)"
- terminal.saveDisk()

5.6 ln — Creacion de enlace simbolico

- LnCommand.execute(["ln", "notas.txt", "/home/users/ana/notas.txt"], terminal)
- targetPath = terminal.resolvePath("notas.txt") -> "/root/notas.txt"
- linkPath = "/home/users/ana/notas.txt"
- target = disk.getFCBByPath(targetPath) -> debe existir; si es null: error "ln: destino no existe"
- Si disk.getFCBByPath(linkPath) != null: error "ln: ya existe"
- parent = disk.getParentFCB(linkPath) -> FCB de /home/users/ana
- AccessControl.checkWrite(parent, user, disk)
- new FCB("notas.txt", user, group, 77, false, linkPath)
 - fcb.setSymlink(true) | fcb.setLinkTarget("/root/notas.txt")
- disk.putFCB(fcb) | parent.addEntry(new DirectoryEntry("notas.txt", id))
- Imprime: "Enlace simbolico: /home/users/ana/notas.txt -> /root/notas.txt"
- terminal.saveDisk()

5.7 su — Autenticacion y cambio de sesion

- SuCommand.execute(["su", "ana"], terminal)

- targetUser = args[1] (o "root" si no se indica usuario)
- Lee contraseña desde Scanner (se oculta visualmente con asteriscos)
- UserManager.authenticate("ana", plainPassword):
 - PasswordHasher.hash(plainPassword) -> SHA-256
 - Compara con users.get("ana").getPasswordHash()
 - Retorna true si coinciden, false si no
- Si false: "Contraseña incorrecta" y retorna sin cambiar sesion
- Si true:
 - terminal.setCurrentUser("ana")
 - user = disk.getUserManager().getUser("ana")
 - terminal.setCurrentDirectory(user.getHomeDirectory())
 - Imprime "Bienvenido, ana."

5.8 useradd — Creacion de usuario

- UseraddCommand.execute(["useradd", "ana"], terminal)
- Verifica que currentUser == "root"; si no: "Permiso denegado"
- Lee interactivamente: nombre completo, contraseña, grupo principal (default: mismo nombre)
- Si el grupo no existe: UserManager.addGroup(groupName)
- UserManager.addUser("ana", fullName, plainPassword, group, "/home/users/ana"):
 - PasswordHasher.hash(plainPassword)
 - new User("ana", fullName, hash, group, homeDir)
 - users.put("ana", user)
 - groups.get(group).addMember("ana")
- Crea directorio home: new FCB("ana", ..., isDirectory=true, "/home/users/ana")
- disk.putFCB(homeFCB) | homeParent.addEntry(entry)
- terminal.saveDisk()

5.9 ls — Listado de directorio

- LsCommand.execute(["ls"], terminal)
- path = args[1] si existe, sino terminal.getCurrentDirectory()
- dirFCB = disk.getFCBByPath(path)
- Si no es directorio: error
- AccessControl.checkRead(dirFCB, user, disk)
- Para cada DirectoryEntry en dirFCB.getEntries():
 - child = disk.getFCB(entry.getFcblid())
 - Imprime: child.typeTag() + permissions + owner + group + size + name
- Con flag -R: para cada entrada que sea directorio, llama listDirectory() recursivamente

6. Manual de Usuario

6.1 Arranque y Entorno de Ejecucion

El VFS Simulator no requiere dependencias externas. Se ejecuta con Java 17 o superior desde la linea de comandos. El archivo de disco miDiscoDuro.fs se crea en el directorio de trabajo actual y se carga automaticamente en cada arranque.

ARRANQUE

```
# Compilar desde el directorio src/
javac -d ../out $(find . -name "*.java")

# Ejecutar
java -cp ../out myFileSystem

# Primer uso: no hay disco, el sistema espera el comando format
[T1] guest@vfs:/# format
    Tamano del disco (ej: 10MB): 10MB
    Nombre del volumen: MiVFS
Disco creado. Sesion iniciada como root.
[T1] root@vfs:/root#
```

El prompt tiene el formato [T<n>] <usuario>@vfs:<directorio>[# o \$], donde # indica root y \$ indica usuario regular. Si no hay sesion iniciada se muestra "guest".

6.2 Gestion de Terminales Multiples

El sistema soporta multiples terminales simultáneas sobre el mismo disco. Los siguientes meta-comandos son gestionados por TerminalManager y NO son delegados a ningun Command individual:

Comando	Descripcion
newterminal	Crea una nueva sesion de terminal (T2, T3, ...) y la activa
switcht N	Cambia el foco a la terminal numero N
terminals	Lista todas las terminales activas con usuario y directorio actual
closeterminal N	Cierra la terminal N. No funciona si es la unica terminal activa

EJEMPLO DE SESION MULTI-TERMINAL

```
[T1] root@vfs:/root# newterminal
[TerminalManager] Nueva terminal T2 creada y activa.
[T2] guest@vfs:/# su ana
Contrasena: ****
Bienvenido, ana.
[T2] ana@vfs:/home/users/ana# terminals
    T1   user=root           cwd=/root
    T2   user=ana            cwd=/home/users/ana  <- ACTIVA
[T2] ana@vfs:/home/users/ana# switcht 1
[TerminalManager] Cambiado a T1 [user=root, cwd=/root]
[T1] root@vfs:/root#
```

6.3 Referencia de Comandos

A continuacion se documenta cada uno de los 27 comandos, con su sintaxis, descripcion de comportamiento, condiciones de error y resultado esperado al ejecutarlo.

format — Sistema

Inicializa el disco virtual. Solicita el tamaño (ej: 10MB) y nombre del volumen. Crea MBR, SuperBloque, árbol de directorios base (/, /root, /home, /home/users), usuario y grupo root e inicia sesión automáticamente. El disco se serializa a miDiscoDuro.fs.

Condiciones de error: Si ya existe disco activo solicita confirmación.

SINTAXIS

```
format
```

RESULTADO ESPERADO

```
[T1] guest@vfs:/# format
      Tamaño: 10MB / Nombre: MiVFS
Disco creado. Sesión iniciada como root.
[T1] root@vfs:/root#
```

exit — Sistema

Cierra la sesión activa del terminal, guarda el disco y termina el proceso si es el último terminal.

SINTAXIS

```
exit
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# exit
Disco guardado. Hasta luego.
```

su — Usuarios y Autenticación

Cambia la sesión al usuario indicado (default: root). Solicita contraseña y la verifica con hash SHA-256. Al autenticarse, actualiza currentUser y currentDirectory al home del usuario destino.

Condiciones de error: Usuario no existe / Contraseña incorrecta / Sin disco activo.

SINTAXIS

```
su [usuario]
```

RESULTADO ESPERADO

```
[T1] guest@vfs:/# su root
Contraseña: ****
Bienvenido, root.
[T1] root@vfs:/root#
```

whoami — Usuarios y Autenticacion

Muestra el usuario activo, nombre completo, grupo principal y directorio home.

Condiciones de error: Requiere sesion iniciada.

SINTAXIS

```
whoami
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# whoami
Usuario : root
Grupo   : root
Home    : /root
```

useradd — Usuarios y Autenticacion

Crea un nuevo usuario solicitando nombre completo, contrasena y grupo principal. Si el grupo no existe lo crea. Crea el directorio home en /home/users/<usuario>.

Condiciones de error: Solo root. El nombre de usuario no debe existir.

SINTAXIS

```
useradd <usuario>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# useradd ana
Nombre completo: Ana Garcia
Contrasena: **** / Grupo [ana]: devs
Usuario creado. Home: /home/users/ana
```

groupadd — Usuarios y Autenticacion

Crea un grupo nuevo en el sistema.

Condiciones de error: Solo root. El grupo no debe existir.

SINTAXIS

```
groupadd <grupo>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# groupadd devs
Grupo 'devs' creado.
```

passwd — Usuarios y Autenticacion

Sin argumento cambia la contrasena del usuario activo; con argumento (solo root) cambia la del usuario indicado. Solicita la nueva contrasena dos veces para confirmacion.

Condiciones de error: Si no es root: verifica contrasena actual. Las dos entradas deben coincidir.

SINTAXIS

```
passwd [usuario]
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# passwd ana
Nueva contraseña: **** / Confirmar: ****
Contraseña actualizada para 'ana'.
```

cd — Navegacion

Cambia el directorio de trabajo. Sin argumento va al home del usuario activo. Acepta rutas absolutas, relativas y ~.

Condiciones de error: Directorio no existe / Sin permiso de ejecucion (x).

SINTAXIS

```
cd [directorio]
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# cd /home/users
[T1] root@vfs:/home/users# pwd
/home/users
```

pwd — Navegacion

Imprime la ruta absoluta del directorio de trabajo actual.

SINTAXIS

```
pwd
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# pwd
/root
```

ls — Navegacion

Lista el contenido del directorio indicado (o el actual). Muestra tipo ([F]/[D]/[L]), permisos, propietario, grupo, tamaño y nombre. Con -R lista recursivamente.

Condiciones de error: Sin permiso de lectura en el directorio.

SINTAXIS

```
ls [-R] [directorio]
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# ls
Tipo Perm Dueno Grupo Bytes Nombre
[F] 77 root root 22 notas.txt
[D] 77 root root 0 proyectos
```

touch — Gestion de Archivos

Crea uno o mas archivos vacios (size=0, bloques=[]). Si el archivo ya existe actualiza su fecha de modificacion.

Condiciones de error: Sin permiso de escritura en directorio padre / Directorio padre no existe.

SINTAXIS

```
touch <archivo> [archivo2 ...]
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# touch a.txt b.txt
[F] 77 root root 0 a.txt
[F] 77 root root 0 b.txt
```

mkdir — Gestion de Archivos

Crea uno o mas directorios (FCB con isDirectory=true, entries vacia).

Condiciones de error: Sin permiso de escritura en directorio padre / Nombre ya existe.

SINTAXIS

```
mkdir <directorio> [dir2 ...]
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# mkdir docs proyectos
Directorio creado: /root/docs
Directorio creado: /root/proyectos
```

rm — Gestion de Archivos

Elimina el archivo liberando sus bloques en el bitmap y eliminando su FCB. Con -R elimina directorios recursivamente. Fuerza cierre en la OFT si el archivo esta abierto.

Condiciones de error: Sin permiso de escritura en padre / Directorio no vacio sin -R / No se puede eliminar /.

SINTAXIS

```
rm [-R] <archivo|directorio>
rm *.ext (wildcards)
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# rm a.txt
Eliminado: /root/a.txt
[T1] root@vfs:/root# rm -R proyectos
Eliminado: /root/proyectos
```

mv — Gestion de Archivos

Mueve o renombra. Si el destino es un directorio existente mueve el archivo dentro. Si no existe, renombra. Actualiza path en el FCB y en todos sus descendientes si es directorio.

Condiciones de error: Origen no existe / Sin permisos en directorios involucrados.

SINTAXIS

```
mv <origen> <destino>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# mv notas.txt apuntes.txt
[F] 77 root root 22 apuntes.txt
```


In — Gestion de Archivos

Crea un enlace simbolico (FCB con isSymlink=true, linkTarget=ruta). cat y less siguen el enlace transparentemente. Si el destino se elimina, el enlace queda roto.

Condiciones de error: Destino no existe / Nombre del enlace ya existe / Directorio padre del enlace no existe.

SINTAXIS

```
ln <destino> <nombre_enlace>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# ln notas.txt /home/users/ana/notas.txt
Enlace simbolico: /home/users/ana/notas.txt -> /root/notas.txt
[T1] root@vfs:/root# cat /home/users/ana/notas.txt
hola mundo
```

cat — Contenido de Archivos

Lee y muestra el contenido completo del archivo. Sigue enlaces simbolicos. Registra apertura y cierre en la OFT.

Condiciones de error: Sin permiso de lectura / No funciona en directorios / "cat: enlace roto" si el destino fue eliminado.

SINTAXIS

```
cat <archivo>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# cat notas.txt
Linea 1: hola mundo
Linea 2: segunda linea
```

less — Contenido de Archivos

Lee el archivo en paginas de 20 lineas. Enter avanza a la siguiente pagina; q cierra el paginador.

Condiciones de error: Sin permiso de lectura / No funciona en directorios.

SINTAXIS

```
less <archivo>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# less notas.txt
--- Pagina 1/1 (lineas 1-2 de 2) ---
Linea 1: hola mundo
Linea 2: segunda linea
--- Fin. [Enter] siguiente [q] salir ---
```

note — Contenido de Archivos

Editor de texto de linea. Si el archivo no existe lo crea; si existe carga su contenido. Cada linea escrita se agrega. Al guardar con :x + Y se reasignan bloques y se actualiza el FCB.

Condiciones de error: Sin permiso de escritura / No funciona en directorios.

SINTAXIS

```
note <archivo>
:x guardar y salir | :q salir sin guardar | :l listar lineas
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# note texto.txt
EDITOR -- texto.txt (:x guardar :q salir :l listar)
> hola mundo
> :x
Guardar? (Y/N): Y
Guardado: /root/texto.txt (10 bytes, 1 lineas, 1 bloques)
```

chmod — Permisos y Propiedad

Cambia los permisos del archivo. El valor es un numero de dos digitos donde cada digito (0-7) suma los bits activos. Imprime el cambio en formato decimal y rwx.

Condiciones de error: Solo el propietario o root. Cada digito debe estar entre 0 y 7.

SINTAXIS

```
chmod <permisos> <archivo>
r=4 w=2 x=1 digito1=propietario digito2=grupo
77=rwxrwx 74=rwxr-- 70=rwx--- 64=rw-r--
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# chmod 74 notas.txt
Permisos actualizados: 77 -> 74 (rwxr--)
```

chown — Permisos y Propiedad

Transfiere la propiedad del archivo al usuario indicado.

Condiciones de error: Solo el propietario actual o root. El usuario destino debe existir.

SINTAXIS

```
chown <usuario> <archivo>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# chown ana notas.txt
Propietario actualizado: root -> ana
```

chgrp — Permisos y Propiedad

Cambia el grupo propietario del archivo.

Condiciones de error: Solo el propietario actual o root. El grupo destino debe existir.

SINTAXIS

```
chgrp <grupo> <archivo>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# chgrp devs notas.txt
Grupo actualizado: root -> devs
```

whereis — Busqueda e Informacion

Busca en toda la tabla de FCBs por nombre exacto o patron wildcard (*, *.ext, prefijo*). Lista todas las rutas que coinciden.

SINTAXIS

```
whereis <nombre|patron>
Ejemplos:  whereis notas.txt   whereis *.txt   whereis doc*
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# whereis *.txt
/root/notas.txt
/root/texto.txt
/home/users/ana/notas.txt
```

viewfcb — Busqueda e Informacion

Muestra todos los atributos del FCB: UUID, nombre, ruta, tipo, propietario, grupo, permisos (decimal y rwx), tamaño, fechas de creacion y modificacion, estado de apertura y bloques asignados.

Condiciones de error: El archivo debe existir.

SINTAXIS

```
viewfcb <archivo>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# viewfcb notas.txt
ID           : a3f7c921-04be-...
Nombre       : notas.txt  Tipo: Archivo
Propietario  : root      Grupo: root
Permisos     : 77 (rwxrwx)
Tamaño       : 22 bytes   Bloques: [5]
Abierto      : No
```

viewfilesopen — Busqueda e Informacion

Muestra la tabla global de archivos abiertos (OFT): UUID, nombre, ruta, usuario que los abrio, fecha/hora de apertura y contador de referencias. NOTA: por la arquitectura de hilo unico del sistema, la OFT siempre aparece vacia en condiciones normales de uso (ver seccion 4.5).

SINTAXIS

```
viewfilesopen
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# viewfilesopen
Total archivos abiertos: 0
(ningun archivo abierto)
```

infofs — Busqueda e Informacion

Muestra estadísticas del sistema de archivos: nombre del volumen, espacio total/usado/libre, bloques totales y libres, cantidad de FCBs, usuarios, grupos y archivos abiertos. Incluye representacion visual del mapa de bits.

Condiciones de error: Requiere disco activo.

SINTAXIS

```
infofs
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# infofs
Volumen      : MiVFS
Total        : 10,485,760 bytes
Libre        : 10,484,736 bytes
Bloques totales: 20480  Libres: 20478
FCBs: 6  Usuarios: 2  Grupos: 2
```

clear — Terminal

Limpia la pantalla mediante la secuencia de escape ANSI \033[H\033[2J.

SINTAXIS

```
clear
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# clear
(pantalla limpia)
```

help — Terminal

Sin argumento lista todos los comandos por categoria. Con argumento muestra descripcion y sintaxis del comando indicado.

SINTAXIS

```
help
help <comando>
```

RESULTADO ESPERADO

```
[T1] root@vfs:/root# help chmod
chmod <permisos> <archivo>
Cambia los permisos de un archivo.
```

7. Bibliografia

- [1]** Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
Capitulos referenciados: Cap. 13 (File System Interface — open(), read(), write(), FCB), Cap. 14 (File System Implementation — estructuras en disco, directorios, Open File Table), Cap. 15 (File System Internals — asignacion de bloques, espacio libre), Cap. 17 (Protection — bits de permiso, ACLs, clases de acceso).
- [2]** Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Pearson.
Capitulos referenciados: Cap. 12 (File Management) — FCB (Tabla 12.1), SuperBloque, mapa de bits de espacio libre, tabla de archivos abiertos, operaciones de apertura y cierre de archivos.
- [3]** Oracle Corporation. (2023). Java SE 17 API Specification.
Referencia para java.io.Serializable, java.io.ObjectOutputStream/ObjectInputStream, java.security.MessageDigest (SHA-256), java.util.UUID, java.util.LinkedHashMap.